

DETAILED COMPREHENSIVE ANSWERS - PART 3

Practical Case Studies, Debugging, and Interview Strategy

GLS DEBUGGING CASE STUDY - THE MULTIPLEXER TIMING DIVERGENCE

Describe in detail a GLS debugging challenge you faced and how you systematically solved it

This is one of my favorite technical stories because it illustrates systematic debugging methodology.

****The Problem**:**

I had completed RTL verification of SERV—comprehensive testbenches, good coverage, all passing. I synthesized the design and generated SDF. When I ran GLS with the identical testbenches, I started seeing divergence. RTL simulation would produce correct results, but GLS simulation would occasionally produce incorrect instruction results. Not consistently—sometimes tests passed, sometimes they failed. This non-deterministic behavior was a red flag.

****Initial Investigation**:**

The first step was to determine if this was a GLS problem or a test issue. I ran the failing test multiple times in both RTL and GLS:

- RTL: Always correct
- GLS: Sometimes correct, sometimes incorrect

This confirmed the issue was GLS-specific. The fact that it was intermittent suggested a timing-related problem, not a functional logic error.

****Reducing the Test Case**:**

GLS is slow (100x slower than RTL), so debugging with full tests is painful. I needed to reduce the failing test case to its minimal form. The failing test contained a sequence of instructions. I binary-searched:

- Run half the instructions → do they fail?
- If yes, try half of that
- If no, try the other half

After binary search, I found that the failure was triggered by a specific instruction: a right-shift instruction (SRA—shift right arithmetic) followed immediately by an ADD

instruction.

The test case:

...

SRA X1, X2, 4 ; Shift X2 right by 4 bits, store in X1

ADD X3, X1, X4 ; Add X1 and X4, store in X3

...

In RTL: X1 correctly contains the shifted value, ADD computes correctly

In GLS: Sometimes X1 has wrong value or the ADD computes with wrong X1

****Waveform Analysis**:**

I loaded both RTL and GLS waveforms in a waveform viewer (DVE). I zoomed into the cycle where X1 should be written:

RTL waveform:

- Cycle N: X1 write signal asserts
- Cycle N: X1 gets updated with correct shift result
- Cycle N+1: X1 is stable, ADD can read it

GLS waveform:

- Cycle N: X1 write signal asserts
- Cycle N+1: X1 starts updating (delayed)
- Cycle N+2: X1 finally settles to correct value
- Meanwhile, ADD in cycle N+1 read the stale value

The issue: X1 wasn't writing fast enough. There was an extra cycle of delay.

****Root Cause Analysis**:**

I instrumented the RTL with signal names to trace the path of the shift result:

1. serv_alu produces alu_rd (shift result)
2. alu_rd is latched by serv_rf_if
3. serv_rf_if drives the register file write

In RTL, this happens "instantly" (zero delay). In GLS with SDF, I needed to check the actual path delays.

Looking at the SDF file:

- ALU output to buffer: 0.8 ns
- Buffer to register file: 1.2 ns
- Register file write delay: 0.5 ns

- Total: 2.5 ns

But my clock period was 10 ns. This should be fine. But wait—the SRA instruction was complex. Let me trace the actual SRA implementation in the ALU.

****The Discovery****:

I found the issue: SRA in a 1-bit serial ALU is complex because you need to know the sign bit (bit 31 for a 32-bit value), but you don't process bit 31 until cycle 31. So the SRA implementation has a complex mux chain:

...

```
// Pseudocode
```

```
shift_in = (shift_right && arith_shift && bit_31) ? 1'b1 : rs2[cycle];
```

...

This created a long combinational path:

- Determine if arithmetic shift (check opcode) → 0.4 ns
- Check if shift_right (check control signal) → 0.3 ns
- Check bit_31 (access register file) → 0.6 ns
- 3-to-1 mux for shift_in → 1.8 ns
- Total: 3.1 ns

Clock period: 10 ns. Seems OK. But wait—let me check if there's another level of logic.

The mux output feeds into the ALU shift chain, which feeds into buffer logic, which feeds into register file write. The full path was:

Bit 31 detection → mux → ALU shift chain → buffer → register file write = 4.7 ns total

Still under 10 ns. But SDF was pessimistic. SDF synthesis extraction sometimes over-estimates certain paths. When GLS ran with SDF, this path was marked as 5.2 ns (SDF pessimism). Add the register file write settling time (another 0.5 ns) and you're approaching the 10 ns limit.

****The Real Issue****:

Actually, after deeper inspection, I found the real problem: a SPECIFIC multiplexer in the shift_in selection had slow logic. The synthesis tool, when optimizing that mux, had selected gates that were fast for general purpose but slow for this specific timing-critical path.

****The Fix****:

I had two options:

1. Modify RTL to simplify the logic path
2. Adjust synthesis constraints to force better optimization of that path

I chose option 2 (simpler and less risky). I added a synthesis directive:

```
...  
// RTL  
(* syn_maxfan = 3 *) // Limit fanout  
shift_in = (shift_right && arith_shift && bit_31) ? 1'b1 : rs2[cycle];  
...
```

This told synthesis to keep the fanout on this signal low, forcing it to use faster buffers. I re-synthesized, regenerated SDF, and re-ran GLS.

The second GLS run: PASSED. Zero divergence from RTL.

****Lessons Learned****:

1. ****Timing issues hide in RTL simulation**** - RTL assumes zero delay; real paths take time
2. ****SDF is accurate but pessimistic**** - It estimates worst case, which can be too conservative
3. ****Systematic debugging beats random fixes**** - Binary search, waveform analysis, root cause analysis
4. ****Document the issue and fix**** - I added a comment explaining the timing-critical path
5. ****Re-verify after fix**** - Always re-run GLS after modifying RTL

This experience is why I emphasize understanding timing and SDF in verification.

VERIFICATION STRATEGIES AND COVERAGE METRICS

Explain your approach to measuring and achieving verification completeness for complex designs

Verification completeness is measured through multiple complementary metrics. No single metric tells the whole story, so I use a dashboard approach.

****Code Coverage (Structural)****:

Code coverage measures which lines of RTL code execute during simulation. Types:

- ****Line coverage****: Did each statement execute? Goal: 95%+
- ****Branch coverage****: Did each if/else branch take? Goal: 95%+
- ****Toggle coverage****: Did each bit in each signal toggle at least once?

For SERV, typical coverage is 98%+ because bit-serial design is very regular—most code paths are exercised by constrained-random testing.

Uncovered code usually indicates either:

1. Dead code (can be removed)
2. Error paths not tested (need directed tests)
3. Features disabled by parameters (acceptable if documented)

****Functional Coverage (Semantic)**:**

Functional coverage measures WHAT the design does, not HOW the code executes. For AES:

- Plaintext pattern coverage: did we test all zero, all one, alternating patterns? → 9 patterns
- Key pattern coverage: same 9 patterns
- Cross-product: $\text{plaintext_pattern} \times \text{key_pattern} = 81$ bins
- Byte-level: for each plaintext byte position, exercise all patterns = $16 \times 9 = 144$ bins

Total functional coverage bins: 200+

Coverage is reported as percentage of bins "hit" (exercised at least once). Goal: 95%+ of bins.

Example: If only 150 of 200 bins are hit, I identify which 50 aren't hit, add directed tests to exercise them, and re-run until 95%+ achieved.

****Coverage-Directed Generation**:**

This is a closed-loop process:

1. Run initial constrained-random stimulus
2. Collect coverage metrics
3. Identify uncovered bins
4. Modify constraint distributions to bias toward uncovered bins
5. Re-run with biased constraints
6. Repeat until convergence

For AES, my distribution was:

- Round 1: 60% random → covers broad input space
- Round 2: Biased toward uncovered patterns (e.g., WALKING_ONE if underreps)
- Round 3: Biased more aggressively
- Repeat until coverage saturates

****Convergence Analysis**:**

A plot of coverage vs. test count should show saturation. Early tests discover many new coverage bins (steep curve). Later tests hit mostly already-covered bins (flat curve). When the curve flattens, you're approaching convergence.

For AES, convergence occurred around 10,000 test cases (coverage went from 80% to 95% in first 5,000, then crawled to 95%+ over next 5,000).

****Bug Discovery Rate**:**

Track how many bugs are found per test case. Early testing finds many bugs (design issues). Middle testing finds fewer (corner cases). Late testing finds almost none (design is mature).

Bug discovery rate = bugs found / test cases run

Expected curve:

- Tests 1-100: 0.5 bugs/test (many issues discovered)
- Tests 100-1000: 0.05 bugs/test (fewer issues)
- Tests 1000+: 0.001 bugs/test (convergence, almost no new bugs)

When bug discovery rate drops below 0.0001 bugs/test, you can sign off (assuming no high-priority open bugs).

****Risk-Based Prioritization**:**

Not all parts of the design are equally risky. I categorize:

- ****High-risk****: Cryptographic operations, constant-time requirements, side-channel sensitive
- ****Medium-risk****: Interface logic, data paths, control flow
- ****Low-risk****: Boring infrastructure, well-understood components

High-risk areas get extra coverage. For AES:

- S-box testing: 100% exhaustive (256 cases)
- Key schedule: Known-vector testing (NIST vectors)
- Round operations: Extensive property-based testing

****Formal Verification Targets**:**

For properties that simulation struggles to validate, use formal verification:

- AES output invertibility: formally prove $\text{AES}(\text{AES}^{-1}(x)) = x$ (tautology, but proves consistency)
- S-box correctness: formally prove all 256 mappings match FIPS spec
- Information flow: prove ciphertext doesn't depend on key bits except through encryption

****Sign-Off Criteria**** (when do you declare "done"?):

1. Code coverage: $\geq 95\%$ line and branch coverage
2. Functional coverage: $\geq 90\%$ of defined bins
3. Known-vector testing: all NIST vectors pass
4. Formal verification: critical properties proven
5. GLS: gate-level simulation passes with zero RTL divergence
6. Bugs: all high-priority bugs fixed, medium/low-priority reviewed
7. Reviewer consensus: verification lead and design lead sign off

Without meeting ALL these criteria, tape-out is not approved.

INTERVIEW POSITIONING AND COMMUNICATION STRATEGY

How would you position your projects and skills to address Qualcomm's Security Hardware DV role?

This is about connecting dots—showing how my diverse background uniquely prepares me for security IP verification.

****The SERV → Security IP Bridge****:

SERV teaches me CPU architecture deeply:

- CSR (Control/Status Register) handling—Qualcomm security IP has CSRs for secure mode control
- Interrupt handling—security cores need sophisticated interrupt mechanisms
- RTL design—I'm not just a testbench writer; I understand hardware design
- GLS debugging—I've debugged real timing issues post-synthesis

I'd say: "My SERV thesis required understanding CPU architecture at a detailed level. Security IP often integrates with CPUs or acts as a specialized processor. Understanding architectural details—CSRs, interrupt handling, state management—directly translates to understanding security IP architecture. Additionally, my GLS debugging experience is crucial. Security IP often has tight timing margins, and post-synthesis verification is where subtle bugs hide. I've debugged those kinds of issues systematically."

****The AES → Security IP Bridge****:

AES demonstrates my ability to handle cryptographic hardware:

- Algorithm understanding—I studied FIPS 197 deeply, not just implemented
- UVM testbench structure—the same approach scales to any crypto core
- Reference models—built behavioral model in SystemVerilog for bit-exact comparison
- Known-vector testing—I used NIST vectors; Qualcomm will use similar standards

- Constant-time awareness—understood why certain implementations matter

I'd say: "While I haven't designed ECC or RSA accelerators, my AES project gives me the foundation. I implemented the full algorithm in both RTL and behavioral model. I understand why certain design choices matter (S-box optimization, constant-time requirements, fault resistance). I know how to verify cryptographic cores using known-vector testing, functional coverage, and reference models. These methodologies transfer directly to other crypto IP."

****Verification Rigor as a Differentiator**:**

I emphasize that verification rigor is my strongest value proposition:

- GLS expertise: Most verification engineers focus on RTL testbenches. I've debugged post-synthesis issues.
- Systematic debugging: When things go wrong, I methodically narrow down root causes (binary search, waveform analysis, instrumentation).
- Completeness thinking: I don't just write tests; I think about coverage metrics, convergence analysis, and sign-off criteria.

I'd say: "In security hardware, 'good enough' doesn't exist. Verification has to be rigorous and complete. My experience shows systematic thinking—when GLS diverged from RTL on SERV, I didn't randomly try fixes; I binary-searched the test case, analyzed waveforms, identified the root cause (timing pessimism in SDF), and fixed it systematically. That's the kind of rigor security verification requires."

****Learning Velocity and Adaptability**:**

I've navigated multiple domains—firmware, ASIC validation, hardware design. Each required learning new concepts and tools. Security is my next specialization:

- Cryptographic algorithms: haven't mastered all yet, but demonstrated ability to learn quickly (AES study)
- Side-channel analysis: studied basics, understand concepts, committed to deepening through hands-on work
- Formal verification: understand methodology, ready to apply in security context

I'd say: "My track record shows I can quickly master new domains. I went from firmware to ASIC verification to hardware design. Security is the next specialization. I've already invested in learning—studied AES, researched side-channel concepts, worked on constant-time design. At Qualcomm, working with security experts would accelerate this learning, and I'm committed to being a dedicated learner."

****Cork Location and Long-Term Commitment**:**

Qualcomm invested in building Cork as a security center. They need people committed to growing that office:

- Based in Cork already
- Not relocating temporarily
- Long-term career perspective
- Genuine interest in contributing to Cork's tech ecosystem

I'd say: "I'm not looking for a temporary role or a stepping stone. I'm based in Cork, and I see this as a long-term opportunity to build my career in security hardware verification at Qualcomm's Ireland office. I'm excited about contributing to the growth of the Cork office and being part of Qualcomm's security IP team for years to come."

****Specific Technical Differentiation****:

Finally, I highlight specific technical skills most verification engineers don't have:

- GLS proficiency: Most verification engineers do RTL only
- Formal verification exposure: RVFI interface, SVA properties
- Hardware design understanding: I've designed RTL, not just tested it
- Power characterization: I've measured and characterized power (SERV stochastic extension)

These aren't common in verification engineers, which makes me stand out.

HANDLING GAPS AND VULNERABILITY

How do you approach admitting knowledge gaps while maintaining credibility?

This is subtle but critical. Admitting gaps is necessary for honesty, but handled poorly, it undermines credibility. Here's my approach:

****The Structure****:

1. ****Acknowledge specifically**** (not generically)

- ✗ "I don't know cryptography" → sounds like you avoided learning
- ✓ "I haven't specialized in post-quantum cryptography yet" → shows specificity

2. ****Demonstrate what you DO know**** (foundation for learning)

- ✗ "I don't know formal verification"
- ✓ "I've used model checking for simple state machines and SVA assertions for timing properties. Information-flow formal verification is more advanced; I'm eager to learn."

3. ****Show proactive learning**** (commitment to growth)

- ✗ Gap without context
- ✓ "I haven't worked with hardware random number generators, but I've studied NIST SP

800-22 statistical tests and understand the verification approach."

4. ****Connect to what you'll do**** (pragmatic plan)

✗ Gap without path forward

✓ "Formal verification for security properties would be new for me, but I understand the concepts. At Qualcomm, working with experts in formal verification would accelerate my learning. I'm committed to mastering these techniques."

****Example Gaps and How I Handle Them****:

Gap: Side-channel analysis experience

Response: "I've researched DPA and timing attacks, understand the threat model, and have designed AES with constant-time in mind. Verification of side-channel resistance through simulation—power trace correlation analysis, timing analysis—is something I'd deepen at Qualcomm working with security specialists."

Gap: ECC accelerator design

Response: "I haven't designed ECC hardware, but I've studied the math (elliptic curve point operations, scalar multiplication algorithms). My AES design experience gives me perspective on crypto accelerator architecture. At Qualcomm, learning ECC specifics would be natural progression from AES."

Gap: Formal verification for information-flow security

Response: "I've used SVA for timing properties and basic model checking for state machines. Information-flow formal verification for proving secrets don't leak is more specialized. I understand the concepts—formalize what 'no information leak' means, use formal tools to prove it—but haven't applied this in practice. This is an area I'm eager to develop at Qualcomm."

****Why This Works****:

- Specificity shows you understand the domain well enough to know what you don't know
- Foundation knowledge shows you're not a blank slate
- Proactive learning shows initiative and commitment
- Concrete path forward shows pragmatism

****What Doesn't Work****:

✗ Bluffing: Pretending to know something you don't. If caught (and you will be), credibility collapses.

✗ Vagueness: "I'm not sure about formal verification" → could mean anything

✗ Defensiveness: "That's not really necessary anyway" → sounds insecure

✗ Overthinking: Spending too much time on the gap. Answer briefly, move on.

****The Mindset**:**

You're not inferior for having gaps. You're a 4+ year engineer with depth in some areas and less depth in others. That's normal. What matters is:

- Awareness (you know what you don't know)
- Honesty (you admit gaps openly)
- Foundation (you have some knowledge in adjacent areas)
- Commitment (you're willing to learn)

At Qualcomm, they want engineers who are honest about gaps and committed to growth. Someone who pretends to know everything is less valuable than someone who's transparent and learning-oriented.